OS-ORANGE-PROBLEM

Name:Shriram Chandrasekar
Section:H
SRN:PES2UG24CS495

Screenshots:
:

```
shriram-cs495@Ubuntu:~/PES2UG24CS495-PES-VCS$ echo $PES_AUTHOR
Shriram<PES2UG24CS495>
```

1a)

```
shriram-cs495@Ubuntu:~/PES2UG24CS495-PES-VCS$ make clean
make all
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -o test_objects test_objects.o object.o -lcrypto
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
/usr/bin/ld: tree.o: in function `tree_from_index':
tree.c:(.text+0x5b6): undefined reference to `index_load'
collect2: error: ld returned 1 exit status
make: *** [Makefile:22: test_tree] Error 1
shriram-cs495@Ubuntu:~/PES2UG24CS495-PES-VCS$ s
```

1b)

```
shriram-cs495@Ubuntu:~/PES2UG24CS495-PES-VCS$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1c)

```
shriram-cs495@Ubuntu:~/PES2UG24CS495-PES-VCS$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

2a)

```
shriram-cs495@Ubuntu:~/PES2UG24CS495-PES-VCS$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

3a)

```
shriram-cs495@Ubuntu:~/PES2UG24CS495-PES-VCS$ ./pes init
Initialized empty PES repository in .pes/
shriram-cs495@Ubuntu:~/PES2UG24CS495-PES-VCS$ echo "hello" > file1.txt
echo "world" > file2.txt
shriram-cs495@Ubuntu:~/PES2UG24CS495-PES-VCS$ ./pes add file1.txt file2.txt
shriram-cs495@Ubuntu:~/PES2UG24CS495-PES-VCS$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  test_sequence.sh
  untracked:  commit.c
  untracked:  test_tree
  untracked:  test_objects
  untracked:  pes.c
  untracked:  test_tree.c
  untracked:  index.h
  untracked:  .gitignore
  untracked:  commit.h
  untracked:  pes.h
  untracked:  object.c
  untracked:  Makefile
  untracked:  index.c
  untracked:  test_objects.c
  untracked:  README.md
  untracked:  tree.h
  untracked:  tree.c

shriram-cs495@Ubuntu:~/PES2UG24CS495-PES-VCS$
```

3b)

```
shriram-cs495@Ubuntu:~/PES2UG24CS495-PES-VCS$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776356176 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776356176 6 file2.txt
shriram-cs495@Ubuntu:~/PES2UG24CS495-PES-VCS$
```

4a)

```
shriram-cs495@Ubuntu:~/PES2UG24CS495-PES-VCS$ ./pes log
commit dfa1092674dd751a964be9089c84ea31d1345d696c63865336f97e1f3c1017db
Author: Shriram <PES2UG24CS495>
Date:   1776357077

    Second commit

commit 4eb7fcc3f12c14eeb9b300d9e114887d3379266b65ac9bd4f0568f35ac77e2d5
Author: Shriram <PES2UG24CS495>
Date:   1776357077

    Initial commit
```

4b)

```
shriram-cs495@Ubuntu:~/PES2UG24CS495-PES-VCS$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/09/b576c2eb51b78fcf9b8493a0909522a75f8ee86df0e926312deeb5fbe7dfb1
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/4e/b7fcc3f12c14eeb9b300d9e114887d3379266b65ac9bd4f0568f35ac77e2d5
.pes/objects/78/3e2e443776e6f6055b056aaf5e8d1ddc91fdc79dc927b01229407d8a84173b
.pes/objects/b4/50a960b8b4cda4c001b2cf95609689f9ee4e65ec663fbc00eb9992c8280c48
.pes/objects/df/a1092674dd751a964be9089c84ea31d1345d696c63865336f97e1f3c1017db
.pes/refs/heads/main
```

4c)

```
shriram-cs495@Ubuntu:~/PES2UG24CS495-PES-VCS$ cat .pes/HEAD
ref: refs/heads/main
shriram-cs495@Ubuntu:~/PES2UG24CS495-PES-VCS$ cat .pes/refs/heads/main
dfa1092674dd751a964be9089c84ea31d1345d696c63865336f97e1f3c1017db
```

Final integration test

```
shriram-cs495@Ubuntu:~/PES2UG24CS495-PES-VCS$ cat .pes/HEAD
ref: refs/heads/main
shriram-cs495@Ubuntu:~/PES2UG24CS495-PES-VCS$ cat .pes/refs/heads/main
dfa1092674dd751a964be9089c84ea31d1345d696c63865336f97e1f3c1017db
shriram-cs495@Ubuntu:~/PES2UG24CS495-PES-VCS$ ./test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 8512c2544b4d... Initial commit

Log after first commit:
commit 8512c2544b4df26561bc3c42893e989208fd494f3adcde1fc04ebf1954b9bdd0
Author: Shriram <PES2UG24CS495>
Date:   1776357257

    Initial commit


--- Second Commit ---
Committed: e01e129e4b67... Update file.txt

--- Third Commit ---
Committed: 7c9e740db734... Add farewell

--- Full History ---
commit 7c9e740db734fff52862c54104d96f303a8d856f646da762ed108238956e6a3f
Author: Shriram <PES2UG24CS495>
Date:   1776357257

    Add farewell

commit e01e129e4b6795b00f763b30901d8b8ff318ec645368b7b3786de0d95d216eed
```

```
--- Second Commit ---
Committed: e01e129e4b67... Update file.txt

--- Third Commit ---
Committed: 7c9e740db734... Add farewell

--- Full History ---
commit 7c9e740db734fff52862c54104d96f303a8d856f646da762ed108238956e6a3f
Author: Shriram <PES2UG24CS495>
Date:   1776357257

    Add farewell

commit e01e129e4b6795b00f763b30901d8b8ff318ec645368b7b3786de0d95d216eed
Author: Shriram <PES2UG24CS495>
Date:   1776357257

    Update file.txt

commit 8512c2544b4df26561bc3c42893e989208fd494f3adcde1fc04ebf1954b9bdd0
Author: Shriram <PES2UG24CS495>
Date:   1776357257

    Initial committial commitEQ


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
7c9e740db734fff52862c54104d96f303a8d856f646da762ed108238956e6a3f

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/7c/9e740db734fff52862c54104d96f303a8d856f646da762ed108238956e6a3f
.pes/objects/85/12c2544b4df26561bc3c42893e989208fd494f3adcde1fc04ebf1954b9bdd0
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/e0/1e129e4b6795b00f763b30901d8b8ff318ec645368b7b3786de0d95d216eed

=== All integration tests completed ===
shriram-cs495@Ubuntu:~/PES2UG24CS495-PES-VCS$
```

Q5.1: A branch in Git is just a file in `.git/refs/heads/` containing a commit hash. Creating a branch is creating a file. Given this, how would you implement `pes checkout <branch>` — what files need to change in `.pes/`, and what must happen to the working directory? What makes this operation complex?

Answer:

To implement `pes checkout <branch>`, the system must update `.pes/HEAD` to point to the target branch by writing `ref: refs/heads/<branch>`. Then it must read the commit hash stored in `.pes/refs/heads/<branch>`.

After obtaining the commit, the working directory must be updated to match the snapshot represented by that commit. This involves reading the commit's tree, recursively traversing it, and recreating all files and directories in the working directory. Existing tracked files must be overwritten, new files created, and files not present in the target tree must be deleted.

This operation is complex because it requires reconstructing an entire directory structure from tree objects, handling file additions and deletions, and ensuring that uncommitted changes are not accidentally lost.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

Answer:

To detect a dirty working directory, the system compares the current working directory with the index. For each file in the index, it checks metadata such as modification time and file size using `stat()`. If either differs from the stored values, the file is considered modified.

Additionally, the system can recompute the hash of the current file and compare it with the hash stored in the index. If the hashes differ, the file has been modified.

If any tracked file is modified and also differs between the current branch and the target branch, the checkout operation must be refused to prevent data loss.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Answer:

In a detached HEAD state, new commits are created normally, but they are not associated with any branch. This means no reference points to these commits, so they can become unreachable once the user switches to another branch.

To recover such commits, the user can create a new branch pointing to the current commit using a command like `pes branch <new-branch>`. This ensures that the commit is referenced and not lost.

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Answer:

To perform garbage collection, the system first identifies all reachable objects. It starts from all branch heads stored in `.pes/refs/heads/` and performs a traversal of commits. For each commit, it marks the commit itself, its parent commits, and the associated tree objects. It then recursively traverses tree objects to mark all referenced blobs and subtrees.

A hash set should be used to store reachable object hashes efficiently, as it provides constant-time lookup.

After marking all reachable objects, the system scans the `.pes/objects/` directory and deletes any object whose hash is not in the reachable set.

For a repository with 100,000 commits and 50 branches, the traversal would visit up to 100,000 commits (many shared across branches) and all associated tree and blob objects. The total number of visited objects could be several hundred thousand depending on the number of files per commit.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Answer:

Running garbage collection concurrently with a commit operation is dangerous because of race conditions. During a commit, new objects are written to the object store, but references to them (such as branch pointers) may not yet be updated.

If garbage collection runs at this moment, it may not see these new objects as reachable and could delete them before the commit is fully completed. This results in corrupted history where a commit references missing objects.

Git avoids this issue by using mechanisms such as temporary object storage, atomic updates, and locking. Objects are only considered for deletion after ensuring that no concurrent operations are in progress, and references are updated atomically to maintain consistency.